
1. Major Stages Involved in Processing a SQL Query

A SQL query passes through several stages before the database produces the final result. The major stages are:

1. Query Parsing and Validation

The SQL query is first analyzed by the **parser**.

It checks:

- SQL syntax
- Table names
- Column names
- Data types
- User permissions
- Validity of SQL commands

For example:

```
SELECT name, department
```

```
FROM Student
```

```
WHERE cgpa > 8.0;
```

The parser verifies that Student, name, department, and cgpa exist and that the query follows SQL syntax.

Effect on response time:

For simple queries, parsing usually takes very little time. However, thousands of dashboard queries can cause repeated parsing overhead. **Prepared statements and query caching** can reduce this overhead.

2. Query Translation

The validated SQL statement is converted into an internal representation, commonly a **relational algebra expression**.

For example:

```
SELECT name
```

```
FROM Student
```

WHERE cgpa > 8.0;

can be represented conceptually as:

$$\pi_{name}(\sigma_{cgpa>8.0}(Student))$$

where:

- σ = Selection
- π = Projection

Effect on response time:

Efficient translation provides the optimizer with a form that can be transformed into a more efficient execution strategy.

3. Query Optimization

This is one of the most important stages.

The **query optimizer** generates different possible execution plans and estimates their costs.

For example, for a join:

SELECT *

FROM Student S

JOIN Placement P

ON S.student_id = P.student_id;

The optimizer may consider:

- Nested-loop join
- Hash join
- Sort-merge join
- Different join orders
- Index scan
- Full table scan

It selects the plan with the lowest estimated cost.

Effect on response time:

A poor execution plan can make a query take seconds or minutes, whereas a good plan can reduce it to milliseconds.

4. Query Execution

The selected execution plan is given to the **query execution engine**.

The engine performs operations such as:

1. Reading data
2. Filtering rows
3. Joining tables
4. Performing aggregation
5. Sorting
6. Producing the final result

For example:

Table Scan

↓

Selection

↓

Join

↓

Aggregation

↓

Sorting

↓

Dashboard Result

Effect on response time:

This is usually the largest contributor to dashboard latency because it involves actual data processing and disk/memory operations.

5. Storage and Data Access

The database accesses data from:

- Memory/cache
- Indexes
- Data pages
- Disk/SSD

If required data is already present in memory, it can be accessed much faster than data requiring disk I/O.

Effect on response time:

Excessive disk I/O significantly increases query execution time. Proper indexing and caching can reduce the number of pages that need to be read.

Overall Query Processing Flow

SQL Query

↓

Parsing & Validation

↓

Translation to Relational Algebra

↓

Query Optimization

↓

Execution Plan

↓

Query Execution

↓

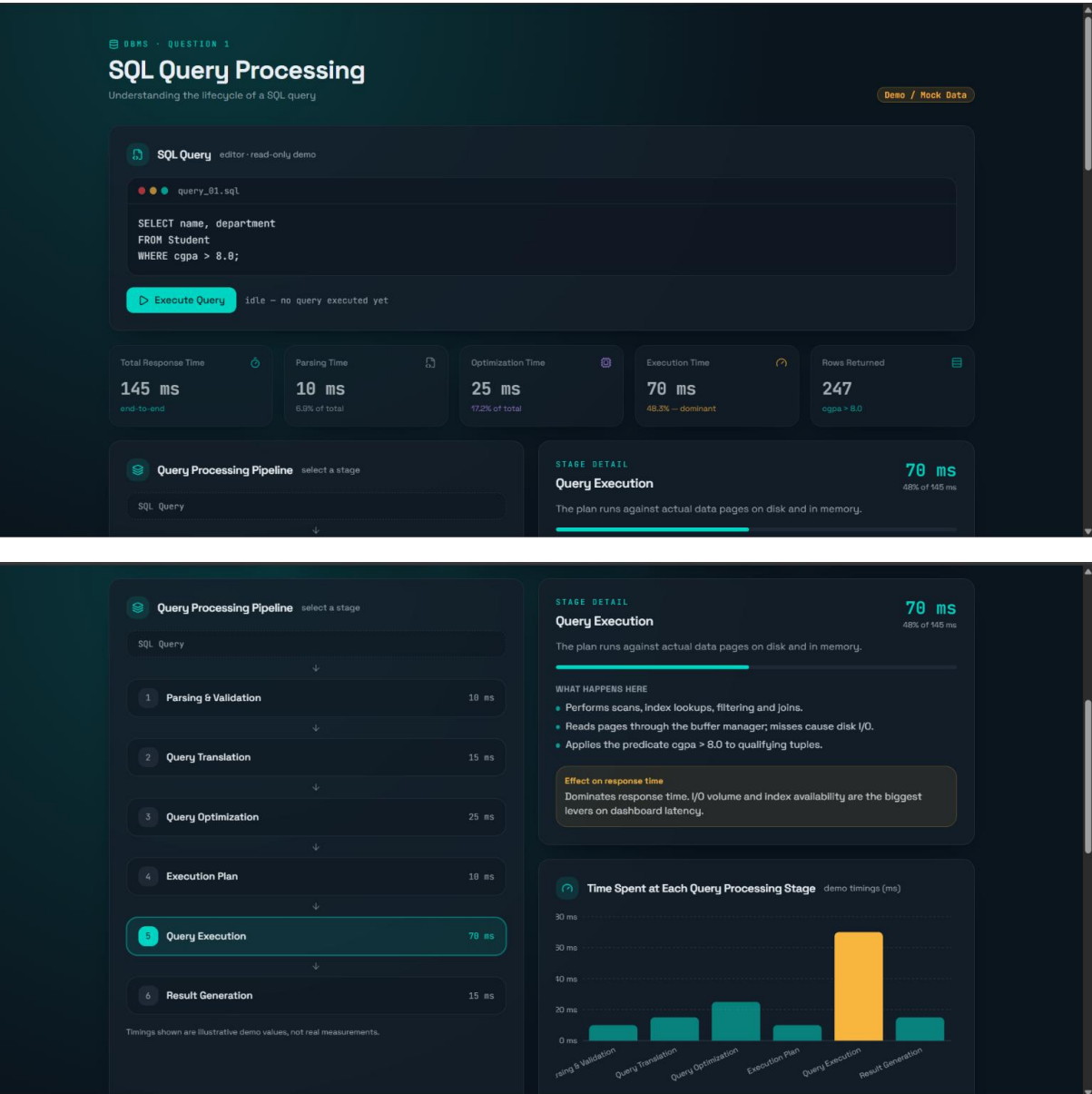
Storage / Index Access

↓

Result

↓

University Dashboard



Relational Algebra Representation

$$\pi(\text{name, department})$$
$$\downarrow$$
$$\sigma(\text{cgpa} > 8.0)$$
$$\downarrow$$
$$\text{Student}$$

SQL is internally transformed into an equivalent relational representation before optimization and execution. The projection π keeps only the requested attributes, the selection σ filters tuples on the predicate, and Student is the base relation scanned by the execution engine.

Query Result

run the query to view results

STUDENT ID	NAME	DEPARTMENT	CGPA
101	Ananya	AI & DS	8.7
102	Rahul	CSE	9.1
103	Priya	ECE	8.4
104	Kavin	IT	8.8
105	Meena	AI & DS	9.0

Page 1 of 50 - mock result set

< Prev

1

2

3

Next >

Conclusion

2. Optimizing a Placement Dashboard Query

```
SELECT S.name,  
       D.department_name,  
       C.company_name,  
       P.package  
FROM Student S  
JOIN Department D  
ON S.department_id = D.department_id
```

JOIN Placement P

ON S.student_id = P.student_id

JOIN Company C

ON P.company_id = C.company_id

WHERE P.placement_year = 2026;

If this query takes several seconds, several optimization techniques can be applied.

Strategy 1: Create Appropriate Indexes

Indexes allow the database to locate records quickly instead of scanning entire tables.

Useful indexes could include:

```
CREATE INDEX idx_student_dept
```

```
ON Student(department_id);
```

```
CREATE INDEX idx_placement_student
```

```
ON Placement(student_id);
```

```
CREATE INDEX idx_placement_company
```

```
ON Placement(company_id);
```

```
CREATE INDEX idx_placement_year
```

```
ON Placement(placement_year);
```

For frequent filtering by year and joining by student:

```
CREATE INDEX idx_placement_year_student
```

```
ON Placement(placement_year, student_id);
```

Benefit

Instead of scanning millions of placement records, the database can directly locate records for the required year.

Strategy 2: Selection Pushdown

Filtering should be performed **as early as possible**.

Instead of joining all placement records and then filtering:

Student

JOIN Placement

JOIN Company

JOIN Department

↓

WHERE placement_year = 2026

the optimizer can first filter:

Placement

↓

WHERE placement_year = 2026

↓

Reduced Placement

↓

Join with Student

↓

Join with Company

↓

Join with Department

Conceptually:

$$\sigma_{year=2026}(Placement)$$

is performed before the joins.

Benefit

Fewer rows participate in joins, reducing CPU, memory, and I/O costs.

Strategy 3: Projection Pushdown

Only the required columns should be retrieved.

Instead of:

SELECT *

use:

```
SELECT S.name,  
       D.department_name,  
       C.company_name,  
       P.package
```

The database can also internally retain only the columns needed for joins and the final result.

Benefit

It reduces:

- Memory consumption
- Data transfer
- I/O
- Processing time

This becomes especially important when tables contain many unnecessary columns.

Strategy 4: Optimize Join Order

The order in which tables are joined can significantly affect performance.

For example:

Placement filtered by year

↓

Student

↓

Department

↓

Company

If only 5% of placement records belong to 2026, filtering them first dramatically reduces the number of rows entering subsequent joins.

Benefit

Smaller intermediate relations result in faster joins.

Strategy 5: Choose an Appropriate Join Algorithm

The optimizer can choose among:

Nested Loop Join

Useful when one relation is small and an index exists on the other table.

Hash Join

Very effective for large equality joins:

`S.student_id = P.student_id`

Sort-Merge Join

Useful when data is already sorted or when large relations need to be joined.

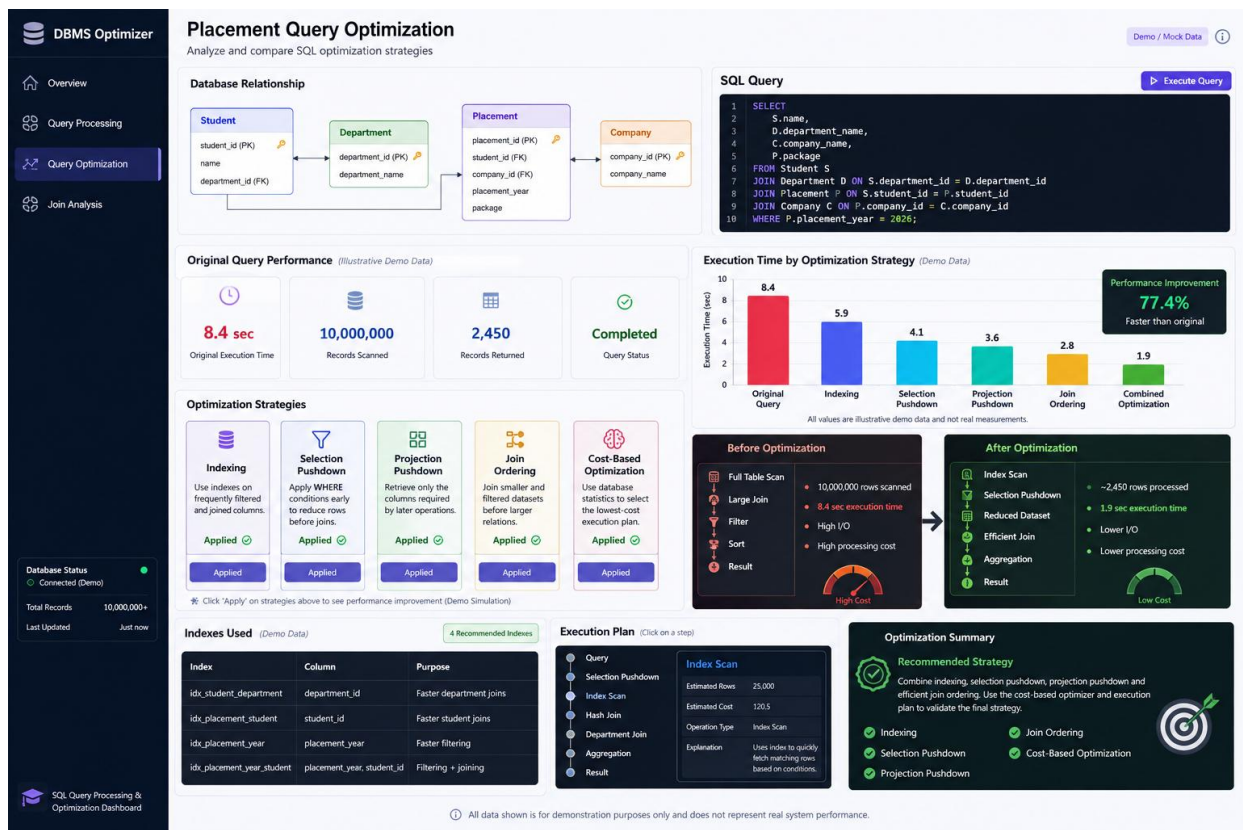
The optimizer should select the algorithm based on table sizes, indexes, available memory, and statistics.

Strategy 6: Cost-Based Optimization

The database optimizer should use statistics such as:

- Number of rows
- Number of distinct values
- Data distribution
- Index selectivity
- Table size

It estimates the cost of different execution plans and chooses the cheapest one.



Conclusion

At least three suitable strategies are:

1. **Indexing**
2. **Selection and projection pushdown**
3. **Join ordering**
4. **Cost-based optimization**
5. **Appropriate join algorithms**
6. **Query/result caching**

Together these techniques can significantly reduce placement-dashboard response time.

3. Comparing Joins for an E-Commerce Dashboard

Suppose an e-commerce dashboard uses:

Customer

↓

Orders

↓

Products

The dashboard wants customer purchase statistics.

For example:

```
SELECT C.customer_id,  
       C.name,  
       COUNT(O.order_id) AS total_orders,  
       SUM(O.quantity * P.price) AS total_purchase  
FROM Customer C  
JOIN Orders O  
      ON C.customer_id = O.customer_id  
JOIN Products P  
      ON O.product_id = P.product_id  
GROUP BY C.customer_id, C.name;
```

Different join types can be used depending on the reporting requirement.

1. INNER JOIN

```
SELECT *  
FROM Customer C  
INNER JOIN Orders O  
      ON C.customer_id = O.customer_id;
```

An INNER JOIN returns only customers who have matching orders.

Example

Customer	Orders
C1	O1
C2	O2

C3 -

Result:

C1 - O1

C2 - O2

C3 is excluded.

Advantage

It processes only matching records, which can be efficient for purchase statistics when customers without purchases are irrelevant.

2. LEFT JOIN

```
SELECT *
```

```
FROM Customer C
```

```
LEFT JOIN Orders O
```

```
ON C.customer_id = O.customer_id;
```

Returns **all customers**, including customers who have not placed an order.

Example:

C1 - O1

C2 - O2

C3 - NULL

Advantage

Useful when the dashboard must show:

- Customers with purchases
 - Customers without purchases
 - Zero-order customers
-

3. RIGHT JOIN

A RIGHT JOIN returns all rows from the right table and matching rows from the left table.

It is less commonly required in this scenario because the same result can usually be expressed more clearly using a LEFT JOIN by changing table order.

4. FULL OUTER JOIN

A FULL OUTER JOIN returns:

- Matching records
- Unmatched customers
- Unmatched orders

It is useful for reconciliation but generally unnecessary for ordinary purchase statistics.

Which Join Should Be Preferred?

For **large datasets**, I would generally prefer an **INNER JOIN** when the dashboard only needs customers who actually made purchases.

Reasons

1. Fewer rows

Customers without orders are eliminated.

2. Smaller intermediate results

Fewer records need to be processed by subsequent joins and aggregations.

3. Better performance

The database has fewer rows to join and aggregate.

4. Appropriate for purchase statistics

If the objective is:

"Calculate purchase statistics for customers who purchased products"

then unmatched customers are not required.

However, if the requirement is:

"Display every customer and show zero purchases for customers with no orders"

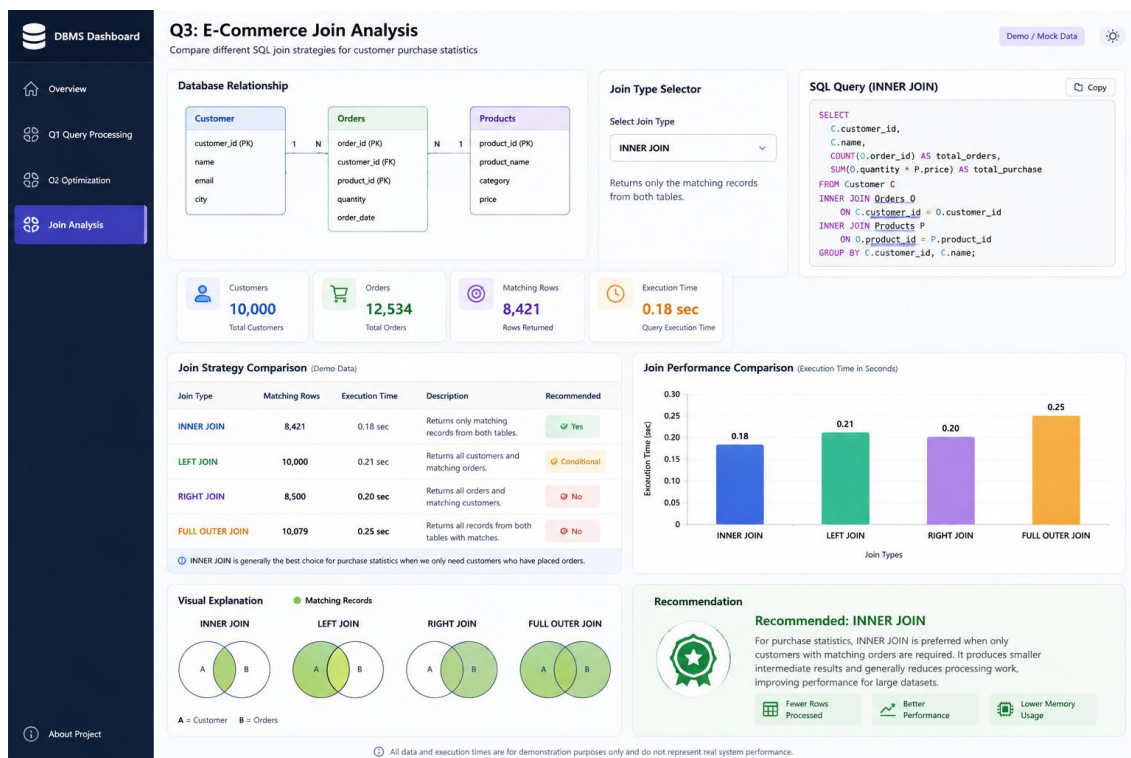
then **LEFT JOIN** is the correct choice.

Conclusion

Join	Use
INNER JOIN	Customers with purchases

Join	Use
LEFT JOIN	All customers including zero purchases
RIGHT JOIN	Less commonly required
FULL OUTER JOIN	Reconciliation / unmatched records

Therefore, for a large-scale purchase-statistics dashboard, **INNER JOIN is preferred when only purchasing customers are required**, because it generally produces smaller intermediate results and reduces processing work.



4. Compulsory Question — University Dashboard Optimization

Problem Statement

A university dashboard takes **18 seconds** to generate a department-wise academic and placement report from a database containing **10 million student-related records**.

The query involves:

- Multiple joins
- Selection
- Projection
- Aggregation

- Sorting

As a database/query-processing engineer, the goal is to identify the causes of poor performance and develop an optimized query-processing strategy.

A. Possible Reasons for Poor Performance

Several factors may cause the 18-second execution time.

1. Large Dataset

The database contains approximately:

10,000,000

student-related records.

Processing millions of records requires considerable:

- CPU
 - Memory
 - Disk I/O
-

2. Missing or Poor Indexes

If columns used in:

WHERE

JOIN

GROUP BY

ORDER BY

are not indexed appropriately, the database may perform full table scans.

For example:

WHERE placement_year = 2026

could require scanning millions of records if placement_year has no useful index.

3. Inefficient Join Order

Joining large tables before filtering them can create huge intermediate relations.

For example:

Student

JOIN Placement

JOIN Company

JOIN Department

↓

WHERE year = 2026

may process unnecessary historical placement records.

4. Unnecessary Columns

Using:

SELECT *

causes unnecessary data to be retrieved and processed.

5. Expensive Aggregation

Operations such as:

COUNT()

SUM()

AVG()

GROUP BY

can consume significant CPU and memory when applied to millions of records.

6. Sorting

A query containing:

ORDER BY

may require a large sort operation.

If the database cannot perform the sorting efficiently using an index, it may need an expensive in-memory or disk-based sort.

7. Outdated Statistics

The cost-based optimizer relies on statistics.

If statistics are outdated, the optimizer may incorrectly estimate:

- Table sizes
- Selectivity
- Join cardinality
- Number of distinct values

and select a poor execution plan.

B. Optimized Query-Processing Strategy

A systematic optimization strategy should be followed.

Step 1: Query Decomposition

Instead of treating the query as one large operation, divide it into logical components.

For example:

Academic Data

↓

Filter relevant students

↓

Placement Data

↓

Filter relevant placement records

↓

Join reduced datasets

↓

Department-wise aggregation

↓

Sorting

↓

Dashboard

This makes it easier to identify expensive operations and optimize individual components.

Step 2: Selection Pushdown

Selection should be performed as early as possible.

Suppose the report is for the year 2026:

WHERE placement_year = 2026

Instead of joining all placement records first, filter them immediately.

Conceptually:

$$\sigma_{placement_year=2026}(Placement)$$

Then join the smaller relation.

Example

10 million Placement Records

↓

placement_year = 2026

↓

500,000 records

↓

JOIN

Instead of joining all 10 million records.

Benefit

This reduces:

- Join cost
- Memory usage
- CPU usage

- Disk I/O
-

Step 3: Projection Pushdown

Only necessary attributes should be carried through the query.

For example:

```
SELECT student_id,  
       department_id,  
       cgpa
```

```
FROM Student;
```

instead of:

```
SELECT *
```

```
FROM Student;
```

For placement:

```
SELECT student_id,  
       company_id,  
       package
```

```
FROM Placement
```

```
WHERE placement_year = 2026;
```

This reduces the size of intermediate relations.

Step 4: Optimize Join Ordering

Join order is extremely important for large databases.

A good strategy is:

Filter Placement

↓

Filter Student

↓

Join Student + Placement

↓

Join Department

↓

Join Company

↓

Aggregate

↓

Sort

The general principle is:

Perform the most selective filters first and join smaller intermediate relations before larger ones whenever possible.

For example, if filtering reduces Placement from 10 million rows to 300,000 rows, it is much better to join those 300,000 rows than the original 10 million.

Step 5: Indexing

Appropriate indexes should be created on columns frequently used for filtering and joining.

For example:

```
CREATE INDEX idx_student_department
```

```
ON Student(department_id);
```

```
CREATE INDEX idx_placement_student
```

```
ON Placement(student_id);
```

```
CREATE INDEX idx_placement_year
```

```
ON Placement(placement_year);
```

For frequent combined filtering and joining:

```
CREATE INDEX idx_placement_year_student
```

```
ON Placement(placement_year, student_id);
```

Depending on the DBMS and workload, indexes may also be useful for:

Student.department_id

Placement.student_id

Placement.company_id

Placement.placement_year

Department.department_id

Company.company_id

Important

Indexes should not be created blindly.

Every additional index:

- Requires storage
- Increases insert/update/delete cost
- May not be useful if the column has poor selectivity

Therefore, indexes should be selected based on the actual workload and execution plan.

Step 6: Cost-Based Optimization

The database's **Cost-Based Optimizer (CBO)** should be used to select the best execution plan.

It estimates the cost of alternatives based on:

- Number of rows
- Index availability
- Selectivity
- Join cardinality
- CPU cost
- I/O cost
- Memory availability

For example, it may compare:

Plan A:

Full Table Scan → Hash Join → Sort

Plan B:

Index Scan → Hash Join → Aggregation

Plan C:

Index Scan → Nested Loop Join → Aggregation

The optimizer chooses the plan with the lowest estimated cost.

Step 7: Examine the Execution Plan

The execution plan is essential for diagnosing the 18-second query.

Depending on the DBMS, commands such as:

EXPLAIN

or

EXPLAIN ANALYZE

can be used.

A plan might reveal:

Full Table Scan

↓

10,000,000 rows

↓

Large Hash Join

↓

Large Sort

↓

Aggregation

This immediately identifies expensive operations.

After optimization, we want something closer to:

Index Scan

↓

Filtered rows

↓

Efficient Hash Join

↓

Aggregation

↓

Small Sort

The actual plan should be compared before and after optimization.

Step 8: Materialized Views

If the department-wise report is requested frequently, recalculating millions of records every time is inefficient.

A **materialized view** can store precomputed results.

For example:

```
CREATE MATERIALIZED VIEW Department_Placement_Summary AS
```

```
SELECT department_id,
```

```
    placement_year,
```

```
    COUNT(*) AS placed_students,
```

```
    AVG(package) AS average_package,
```

```
    MAX(package) AS highest_package
```

```
FROM Placement
```

```
GROUP BY department_id, placement_year;
```

The dashboard can then query the smaller precomputed structure.

```
SELECT *
```

```
FROM Department_Placement_Summary
```

```
WHERE placement_year = 2026;
```

Benefit

Instead of repeatedly processing millions of records:

10 million records



JOIN



GROUP BY



SORT

the dashboard accesses already aggregated results.

Trade-off

Materialized views must be refreshed when underlying data changes.

Therefore, they are especially suitable when:

- Reports are requested frequently
- Data changes less frequently
- Fast dashboard response is important

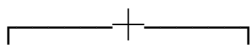
Step 9: Parallel Processing

With 10 million records, the DBMS can use **parallel query execution** if supported.

The work can be divided among multiple CPU cores.

For example:

10 Million Records



CPU 1 CPU 2 CPU 3 ... CPU N



Partial Results



Merge

↓

Final Report

Parallelism can be applied to:

- Table scans
- Joins
- Aggregations
- Sorting

Benefit

Multiple processors work simultaneously, reducing elapsed execution time.

Caution

Excessive parallelism can increase:

- CPU contention
- Memory consumption
- Coordination overhead

Therefore, the degree of parallelism should be selected experimentally.

C. Optimized Processing Pipeline

The complete strategy can be represented as:

SQL Query

↓

Parse & Validate

↓

Query Decomposition

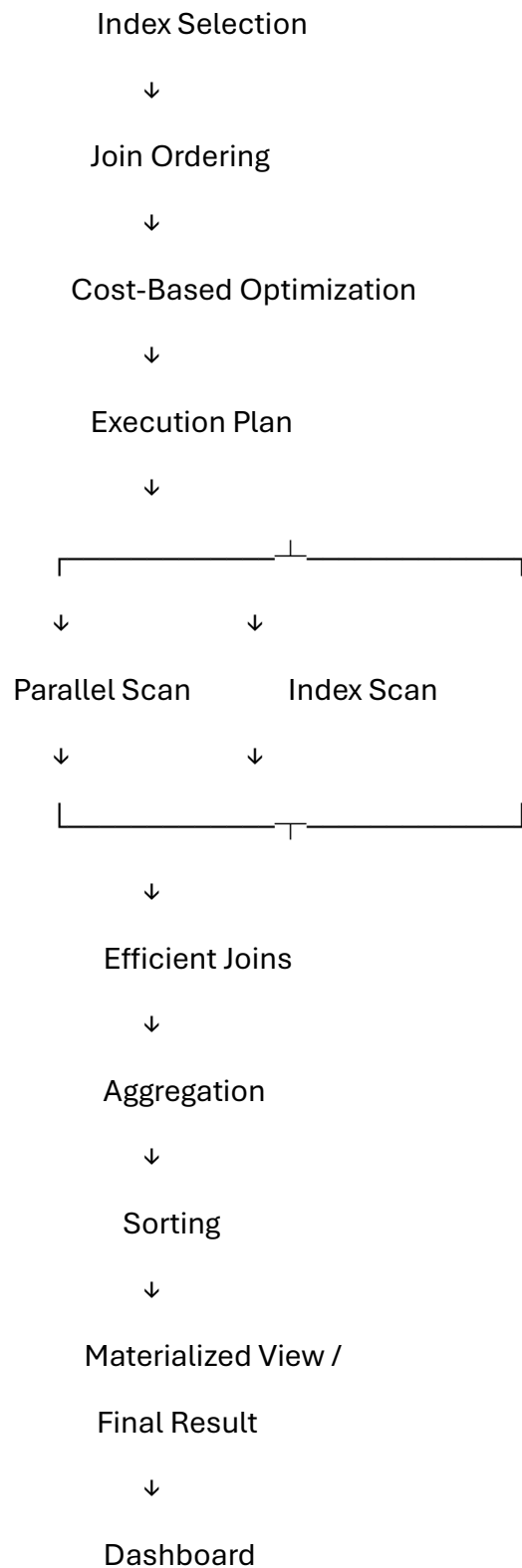
↓

Selection Pushdown

↓

Projection Pushdown

↓



D. Example Optimized Query

A simplified optimized query could be:

SELECT

```

D.department_name,
COUNT(DISTINCT S.student_id) AS total_students,
COUNT(DISTINCT P.student_id) AS placed_students,
AVG(P.package) AS average_package
FROM Student S
JOIN Department D
    ON S.department_id = D.department_id
LEFT JOIN Placement P
    ON S.student_id = P.student_id
    AND P.placement_year = 2026
GROUP BY D.department_name
ORDER BY D.department_name;

```

The important optimization here is that:

```
P.placement_year = 2026
```

is associated directly with the Placement relation before unnecessary placement records participate in the join.

Appropriate indexes could include:

```

CREATE INDEX idx_placement_year_student
ON Placement(placement_year, student_id);

```

and:

```

CREATE INDEX idx_student_department
ON Student(department_id);

```

The exact indexes should ultimately be validated against the DBMS execution plan and workload.

E. Experimental Proof of Improvement

The optimization should **not simply be assumed to work**. It must be experimentally measured.

1. Establish a Baseline

First execute the original query several times.

Suppose the measurements are:

Query	Execution Time
-------	----------------

Original – Run 1	18.4 s
------------------	--------

Original – Run 2	17.9 s
------------------	--------

Original – Run 3	18.2 s
------------------	--------

Average	18.17 s
----------------	----------------

Also record:

- CPU usage
- Disk I/O
- Memory usage
- Rows processed
- Rows returned
- Execution plan

2. Apply One Optimization at a Time

Test:

Experiment A — Indexing

Create the required indexes and execute the query again.

Experiment B — Selection Pushdown

Filter records before joins.

Experiment C — Projection Pushdown

Retrieve only required columns.

Experiment D — Join Ordering

Change the order of joins or allow the optimizer to select a better order.

Experiment E — Materialized View

Compare the original query against the precomputed summary.

Experiment F — Parallel Processing

Test different degrees of parallel execution.

3. Compare Execution Plans

Use:

EXPLAIN ANALYZE

before and after optimization.

Compare:

- Estimated cost
 - Actual execution time
 - Number of rows scanned
 - Number of rows processed
 - Join algorithms
 - Index usage
 - Sort operations
-

4. Example Experimental Results

A hypothetical result could be:

Version Optimization		Time
Original None		18.0 s
V1	Indexing	11.5 s
V2	Selection + Projection Pushdown	8.2 s
V3	Better Join Ordering	6.4 s
V4	Cost-Based Plan + Indexing	4.7 s
V5	Materialized View	1.2 s
V6	Materialized View + Parallel Processing	0.8 s

These numbers are **illustrative**; actual results must be measured on the university database.

The percentage improvement can be calculated as:

$$\text{Improvement} = \frac{\text{Original Time} - \text{Optimized Time}}{\text{Original Time}} \times 100$$

If the original time is 18 seconds and optimized time is 0.8 seconds:

$$\frac{18 - 0.8}{18} \times 100 = 95.56\%$$

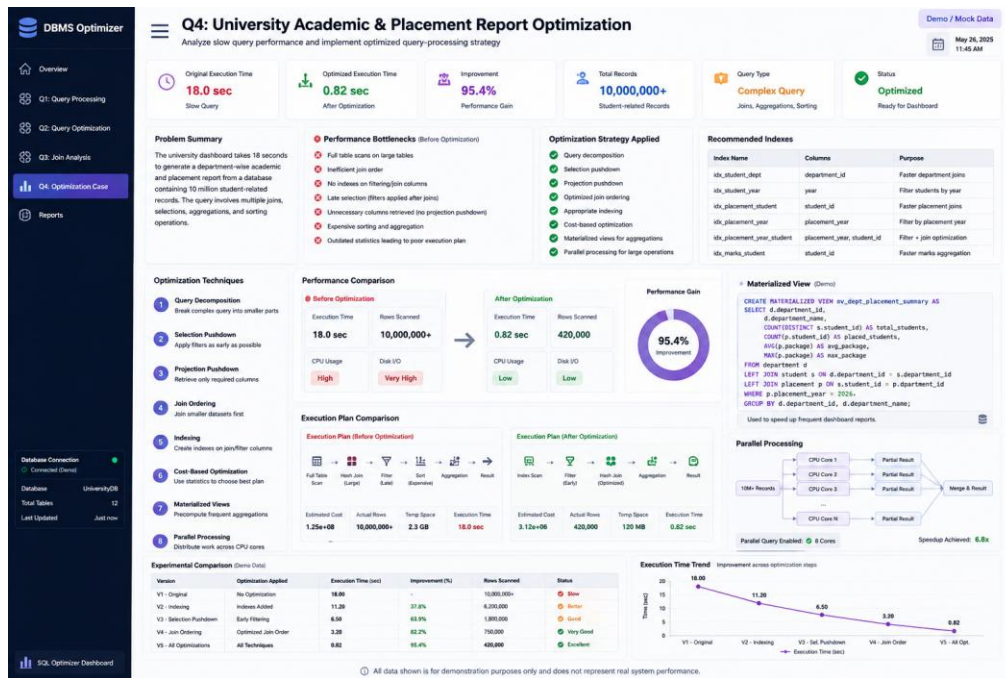
Thus, the optimized solution would provide approximately **95.6% reduction in execution time** in this hypothetical experiment.

Final Conclusion for Q4

The 18-second response time is likely caused by the large volume of data, inefficient joins, unnecessary rows and columns, missing indexes, expensive aggregation and sorting, inaccurate statistics, or a poor execution plan.

The optimized strategy should therefore combine:

1. **Query decomposition**
2. **Selection pushdown**
3. **Projection pushdown**
4. **Efficient join ordering**
5. **Appropriate indexing**
6. **Cost-based optimization**
7. **Execution-plan analysis**
8. **Materialized views for frequently requested reports**
9. **Parallel processing where beneficial**



Finally, the improvement should be experimentally demonstrated by comparing **execution time, CPU usage, I/O, rows processed, and execution plans** before and after optimization. This provides objective evidence that the optimized query-processing strategy is superior to the original 18-second implementation.